

PORTADA

Técnicas de Programación

Unidad 7 — Desarrollo Web: Framework Spring Boot: Introducción y API REST

Arquitectura de Spring Boot, anotaciones clave y creación de un controlador REST

Universidad de Antioquia · Ingeniería de Sistemas · 2508307



TRANSICIÓN

De consumir una API a construirla

En las últimas seis sesiones aprendiste a maquetar con HTML y CSS3, y a darle comportamiento a una página con JavaScript — incluyendo `fetch()` para **consumir** una API REST y mostrar sus datos en pantalla. Siempre asumiste que, del otro lado de esa URL, **algo** respondía: recibía la petición HTTP, buscaba o guardaba información, y devolvía JSON.

Hoy construimos ese "algo": el servidor. Y lo hacemos en **Java**, el lenguaje que ya conoces a fondo del resto del curso, usando el framework más usado del mundo Java empresarial: **Spring Boot**.

CONCEPTO

Qué es un framework

Un **framework** es un conjunto de herramientas, librerías y convenciones que ya resuelven los problemas **comunes a cualquier aplicación de su tipo** — un servidor HTTP que reciba conexiones, un mecanismo de enrutamiento que decida qué código atiende cada URL, la serialización automática de objetos a JSON, manejo de errores, etc.

La idea central: en vez de reinventar esa infraestructura en cada proyecto, el framework ya la trae resuelta y probada — así el desarrollador puede enfocarse en la **lógica de negocio**, que es lo que realmente distingue a su aplicación.



CONCEPTO

Spring y Spring Boot

Spring es el ecosistema de frameworks para Java más usado en el desarrollo de aplicaciones empresariales: manejo de dependencias, acceso a datos, seguridad, servicios web, entre muchos otros módulos. Es potente, pero configurarlo manualmente (archivos XML, servidores externos, dependencias sueltas) es lento y propenso a errores.

Spring Boot es la forma moderna de usar Spring: simplifica radicalmente esa configuración con tres ideas clave.

Idea	Qué resuelve
Arranque rápido	Un proyecto ejecutable en minutos, sin configurar servidores externos
Servidor embebido	Trae su propio servidor HTTP (Tomcat) empaquetado, no hay que instalar uno aparte
Convención sobre configuración	Valores por defecto sensatos; solo configuras lo que quieres cambiar



CONCEPTO

Inversión de control: Spring crea los objetos por ti

Normalmente, cuando un objeto necesita a otro en Java, lo instancias tú mismo con `new`. Spring Boot invierte ese control: es el **framework** quien crea, configura y entrega los objetos que tu aplicación necesita — a eso se le llama **inyección de dependencias**, y a esos objetos gestionados por Spring se les llama **beans**.

No es un concepto nuevo: es la aplicación práctica del principio de **Inversión de Dependencias** (la "D" de SOLID, Unidad 3) — las clases dependen de abstracciones, y algo externo a ellas decide qué implementación concreta entregarles. Ese "algo externo" es Spring.



CONCEPTO

Spring Initializr: el punto de partida

Spring Initializr es un generador web (o integrado en el IDE) que arma la estructura base de un proyecto Spring Boot: eliges lenguaje, versión de Java, gestor de dependencias (Maven/Gradle) y las **dependencias** que necesitas — para una API REST, basta con **Spring Web**.

El proyecto generado trae esta estructura:

- `src/main/java/...` — todo el código fuente de la aplicación
- `src/main/resources/application.properties` — configuración (puerto, nombre, conexión a datos)
- Una **clase principal**, anotada con `@SpringBootApplication`, lista para ejecutar



CONCEPTO

La clase principal

`@SpringBootApplication` marca la clase de arranque: le dice a Spring "desde aquí construye y configura toda la aplicación" (escanea componentes, habilita auto-configuración, etc.). El método `main` llama a `SpringApplication.run(...)`, que levanta el servidor embebido y deja la aplicación escuchando peticiones.

```
@SpringBootApplication
public class TareasApplication {

    public static void main(String[] args) {
        SpringApplication.run(TareasApplication.class, args);
    }
}
```

Ejecutar esta clase levanta el servidor — normalmente en `http://localhost:8080` .

CONCEPTO

Ciente-servidor, ahora del lado del servidor

Al ejecutar la clase principal, Spring Boot levanta un **servidor HTTP embebido** (por defecto, **Tomcat**) que queda escuchando conexiones en un puerto. Es exactamente la misma arquitectura cliente-servidor y los mismos verbos HTTP (GET , POST , etc.) vistos en la primera sesión de esta unidad — solo que ahora tú escribes el código que **responde** del lado del servidor, no el que hace la petición desde el navegador.

Cuando llega una petición HTTP, Spring debe decidir **qué código Java la atiende**: eso es responsabilidad de los **controladores**.



CONCEPTO

@RestController : la clase que responde peticiones

Una clase anotada con `@RestController` le indica a Spring que es un **controlador REST**: sus métodos atienden peticiones HTTP y lo que devuelven se serializa **automáticamente** (normalmente a JSON) en el cuerpo de la respuesta, sin que tengas que convertir nada a mano.

Dentro de esa clase, cada método se conecta a una ruta y un verbo HTTP específicos usando otra anotación: `@GetMapping` , `@PostMapping` , etc.



CONCEPTO

@GetMapping : exponer datos con GET

`@GetMapping("/ruta")` mapea una petición GET a esa ruta con el método que sigue. El objeto o la lista que el método retorna se convierte automáticamente en la respuesta JSON.

```
@RestController
public class TareaController {

    private List<Tarea> tareas = new ArrayList<>(List.of(
        new Tarea(1L, "Estudiar Spring Boot", false),
        new Tarea(2L, "Entregar taller", false)
    ));

    @GetMapping("/tareas")
    public List<Tarea> listarTareas() {
        return tareas;
    }
}
```

Una petición GET /tareas responde el arreglo JSON con las dos tareas.

CONCEPTO

@PathVariable y @RequestParam : recibir datos desde la URL

Ambas anotaciones toman datos de la URL de la petición, pero de lugares distintos:

Anotación	Dónde va el dato	Ejemplo de URL
@PathVariable	Parte del path , identifica un recurso	/tareas/{id}
@RequestParam	Query string , filtra o parametriza	/tareas/buscar?completada=true

```
@GetMapping("/tareas/{id}")
public Tarea buscarPorId(@PathVariable Long id) { /* ... */ }

@GetMapping("/tareas/buscar")
public List<Tarea> buscarPorEstado(@RequestParam boolean completada) { /* ... */ }
```

CONCEPTO

@PostMapping y @RequestBody : recibir un objeto JSON

@PostMapping mapea peticiones POST (crear un recurso). @RequestBody le dice a Spring que tome el **cuerpo JSON** de la petición y lo convierta automáticamente en un objeto Java.

```
@PostMapping("/tareass")
public Tarea crearTarea(@RequestBody Tarea nuevaTarea) {
    tareas.add(nuevaTarea);
    return nuevaTarea;
}
```

En la Unidad 4 serializabas objetos a JSON **a mano** con Gson. Aquí Spring hace ese trabajo por ti, en ambas direcciones, usando internamente una librería llamada **Jackson** — el mismo concepto de serialización, ahora automatizado por el framework.



CONCEPTO

Ejemplo completo: un controlador con GET y POST

```
@RestController
@RequestMapping("/productos")
public class ProductoController {

    private List<Producto> productos = new ArrayList<> (List.of(
        new Producto(1L, "Teclado", 120000),
        new Producto(2L, "Mouse", 45000)
    ));

    @GetMapping
    public List<Producto> listar() {
        return productos;
    }

    @PostMapping
    public Producto crear(@RequestBody Producto nuevo) {
        productos.add(nuevo);
        return nuevo;
    }
}
```

`@RequestMapping("/productos")` en la clase fija el prefijo común de ruta para todos sus métodos.

INTERACCIÓN

Diseñen la mini-API de "estudiantes"

En parejas: en papel o pseudocódigo, diseñen las rutas y verbos HTTP de un mini-API REST para el recurso "**estudiantes**".

Para cada endpoint definan: la **ruta**, el **verbo HTTP** y qué **devuelve o recibe**.

- Listar todos los estudiantes
- Consultar un estudiante por su ID
- Buscar estudiantes por programa académico
- Registrar un estudiante nuevo

8 minutos · al final, una pareja comparte su diseño con el resto del curso

SÍNTESIS

Lo que hay que llevarse de hoy

1. Un **framework** resuelve la infraestructura común para que te enfoques en la lógica de negocio.
2. **Spring Boot** simplifica Spring: arranque rápido, servidor embebido, convención sobre configuración.
3. Spring gestiona tus objetos (**beans**) por ti — inyección de dependencias, la práctica de la "D" de SOLID.
4. `@SpringBootApplication` + `SpringApplication.run(...)` levantan la aplicación.
5. `@RestController` marca una clase que responde peticiones HTTP y serializa a JSON automáticamente.
6. `@GetMapping` / `@PostMapping` mapean rutas; `@PathVariable` , `@RequestParam` y `@RequestBody` reciben datos de la URL o del cuerpo de la petición.



CIERRE

Antes de la próxima sesión

- Genera un proyecto en [Spring Initializr](#) con la dependencia **Spring Web** y ejecútalo localmente.
- Trae listo el diseño de la mini-API de "estudiantes" del ejercicio de hoy — se retoma para implementarlo.

Próxima sesión: arquitectura en capas, persistencia con Spring Data JPA — y cierre de la Unidad 7.

